FA23

Dynamic Branch Prediction with Perceptrons

@nebu

Outline

Overview of Branch Prediction

Overview of Perceptrons and Related Math

The Paper

Section 1

Overview of Branch Prediction

Pipelining Requires *Prediction*

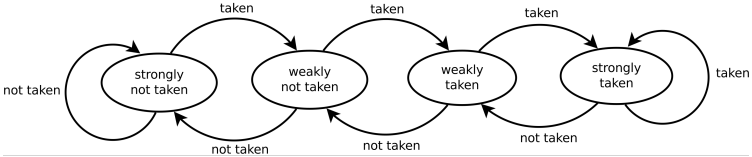
- Imagine a conditional branch instruction enters the pipeline.
- Until we figure out whether to branch (this requires going through a couple of pipeline stages…) what do we do?
- We can stall, but that's wasteful.
- Better to "predict" which way the branch will go to figure out where to issue next instructions from.

Ways to Predict

- "Static": always predict branches are taken (or untaken). Or some other static scheme, like always take backward branches, never forward branches, etc. etc.
- "Random": flip a coin to decide whether to take the branch. Makes timing nondeterministic.
- "Dynamic": much better, learns with the program at runtime.

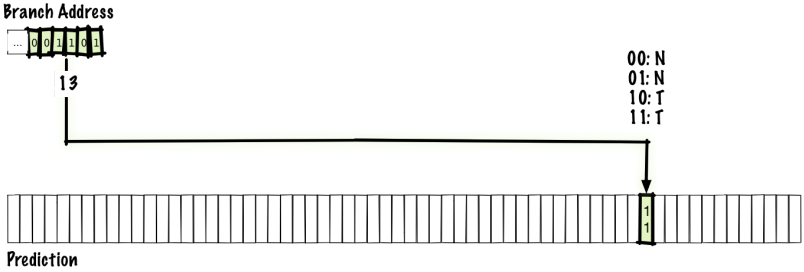
Dynamic Methods: One Level Saturating Counters

- One level, one bit: record the last outcome of the branch, assume the next one will go the same way.
- One level, two bit saturating counter.



- Better since *two* branches that go the other way are required to change state.

One-Level Indexing



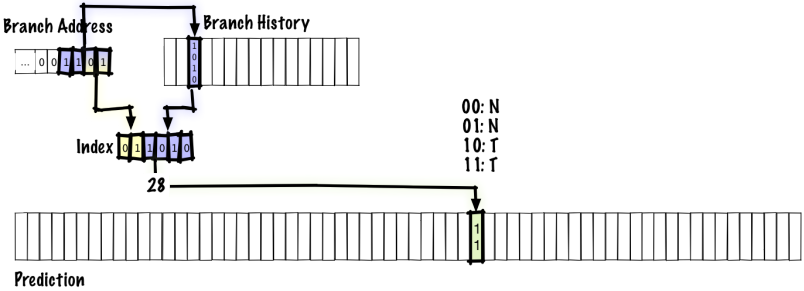
- There's a table of predictions indexed by the address of the instruction for predicting each branch.[1]

[1]Image credit: `https://danluu.com`

Dynamic Methods: Two Level Local

- Key idea is to catch repeating patterns of branches, for example: `100100100100`...
- Requires remembering the past n directions of the branch.
- Have 2^n two bit saturating counters per branch, one for each possible history pattern (it'll be in $\{0, 1\}^n$).

Two Level Local Diagram



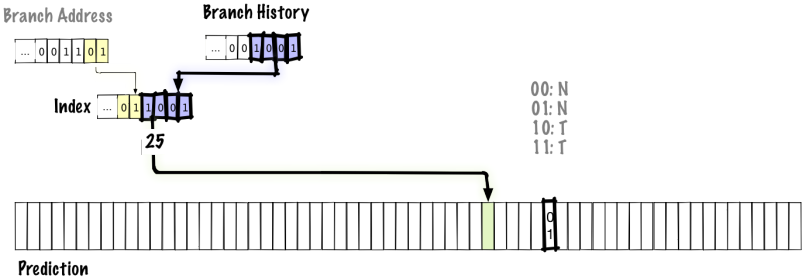
[2]

[2]Image credit: `https://danluu.com`

Local vs. Global Prediction

- Local: maintain a history for each individual branch instruction.
- Global: use a *global* history table instead of one local to each branch.
  - If branch directions are correlated with each other, performs better, else caused contamination between branches.

Two Level Global Diagram (gselect)



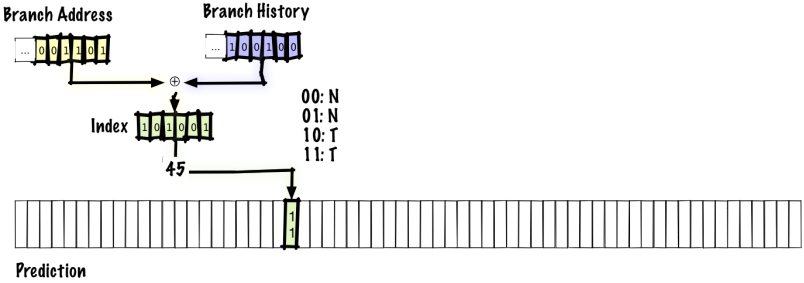
[3]

[3]Image credit: https://danluu.com

gshare

- `gselect`, for a fixed-size prediction table, makes you choose between the number of branch PC bits and the number of global history bits.
- Want both: low interference between branches, and long histories to catch complex patterns.
- To get both, use `gshare`: hash them together with an XOR.

gshare Diagram



Hybridization (McFarling predictors)

- Have multiple prediction mechanisms.
- Pick the predictor that's made the best predictions in the past. (meta-prediction)
- Or use a majority function on predictor outputs.

Lots of other schemes...

- Many schemes attempt to reduce destructive interference due to branch aliasing.
- Others do more involved things.
- bi-mode, gskew, YAGS, TAGE...
- Today, we'll talk about a perceptron-based prediction mechanism.

Section 2

Overview of Perceptrons and Related Math

Dot Products

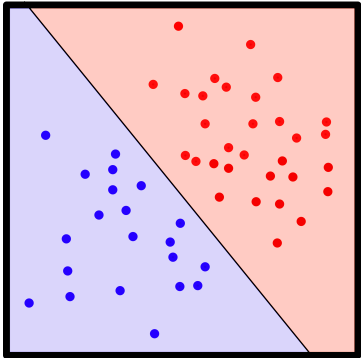
- A vector can be thought of as a list of numbers:[4]
  $\vec{a} = (a_1, a_2, a_3, \ldots, a_n)$ $\vec{b} = (b_1, b_2, b_3, \ldots, b_n)$
- A dot product of two vectors $\vec{a}$ and $\vec{b}$ is defined as:

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

[4] Math majors: this is an oversimplification, since this is only true for the special case of coordinate spaces…vectors in general don't need to be "lists".

Linear separability in $\mathbb{R}^2$

Two sets of points (colored say red and blue) on the plane are linearly separable if there exists one line that separates the red points from the blue points.



Linear separability, generalized

- Generalizing to any n dimensional space, sets of points are linearly separable if there is a hyperplane that separates them.
- In the language of convex hulls, two sets of points are linearly separable if their convex hulls are disjoint.

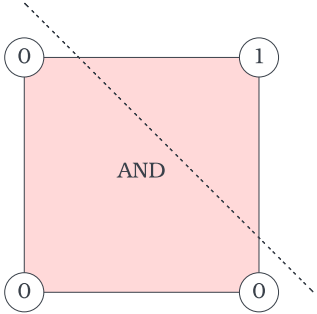
Linear separability, formal definition

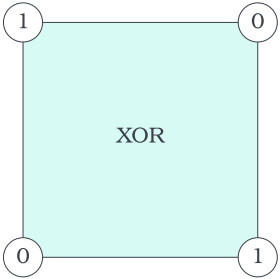
Definition

For sets of points X_0 and X_1 in some n dimensional Euclidean space, X_0 and X_1 are linearly separable if there are $n+1$ real numbers $\vec{w} = (w_i)_{i \in [1,n]}$ and k, such that for all $\vec{x} \in X_0$, $\vec{x} \cdot \vec{w} > k$ and for all $\vec{x} \in X_1$, $\vec{x} \cdot \vec{w} < k$.

Linear separability of Boolean functions

A Boolean function $f : \{0,1\}^n \to \{0,1\}$ is linearly separable if the hypercube of dimension n with vertices assigned to outputs of the function is linearly separable.





Linear Classifiers

- A linear classifier is a function that, given two sets of data and a new data point, decides which set the data point belongs to.
- Given a vector of weights $\vec{w}$ and a feature vector $\vec{x}$, the classifier computes an output score y, given by

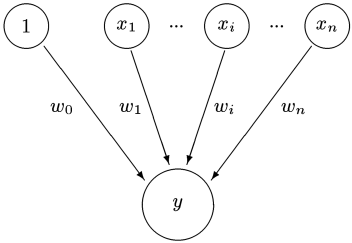
$$y = f(\vec{w} \cdot \vec{x})$$

- Typically f is a threshold function, which is simply a unit step function shifted by θ:

$$f(\vec{w} \cdot \vec{x}) = \begin{cases} 1, & \vec{w} \cdot \vec{x} > \theta \\ 0, & \text{otherwise} \end{cases}$$

Perceptrons

- A perceptron is a *learning* linear classifier.
- It outputs a classification, but also uses the *correct* answer to update the weight vector $\vec{w}$.
- It can be viewed as an artificial neuron whose activation function is the unit step function.



Key Insight: Branch Prediction is a Boolean Function

- The paper shows that perceptrons can be successfully used to learn certain branches.
- …with a low hardware overhead.
- Note that a primary limitation of perceptrons is that they can only learn linearly separable functions (i.e., a perceptron couldn't correctly learn the XOR function).

Section 3

The Paper

Citation

D.A. Jimenez and C. Lin, “Dynamic branch prediction with perceptrons,” *Proceedings HPCA Seventh International Symposium on High-Performance Computer Architecture*, Monterrey, Mexico, 2001, pp. 197-206, doi: 10.1109/HPCA.2001.903263.